

Angular Moderno

32 - Refatoração do Projeto

2026

SENAI - CRTI

32 — Refatoração do Projeto

Nesta aula vamos realizar uma **refatoração estrutural** no projeto com foco em boas práticas de arquitetura e organização de código.

Organização do Código

Estrutura clara e padronizada

Separação de Responsabilidades

Cada componente com seu papel

Eliminação de Duplicação

Lógica centralizada e reutilizável

Fluxo de E-commerce Real

Catálogo → Carrinho → Checkout

 **Objetivo principal:** Corrigir o fluxo do sistema para funcionar como um e-commerce real: **catálogo → carrinho → checkout**





Problema Atual do Projeto

Antes desta refatoração, o projeto apresenta uma série de problemas estruturais que comprometem a manutenção e a escalabilidade.

Problemas Identificados

- O carrinho está espalhado em múltiplas telas
- A lógica do carrinho aparece na lista de produtos
- O checkout também exibe e manipula itens diretamente
- Existe duplicação de responsabilidade entre componentes

Impacto no Projeto

Difícil Manutenção

Código acoplado e confuso

Alto Risco de Bugs

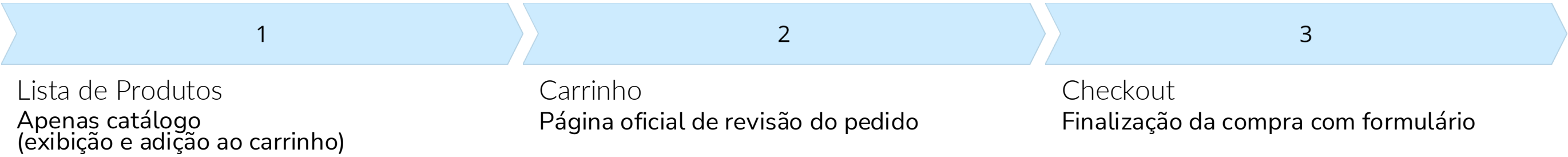
Lógica duplicada gera inconsistências

Arquitetura Inconsistente

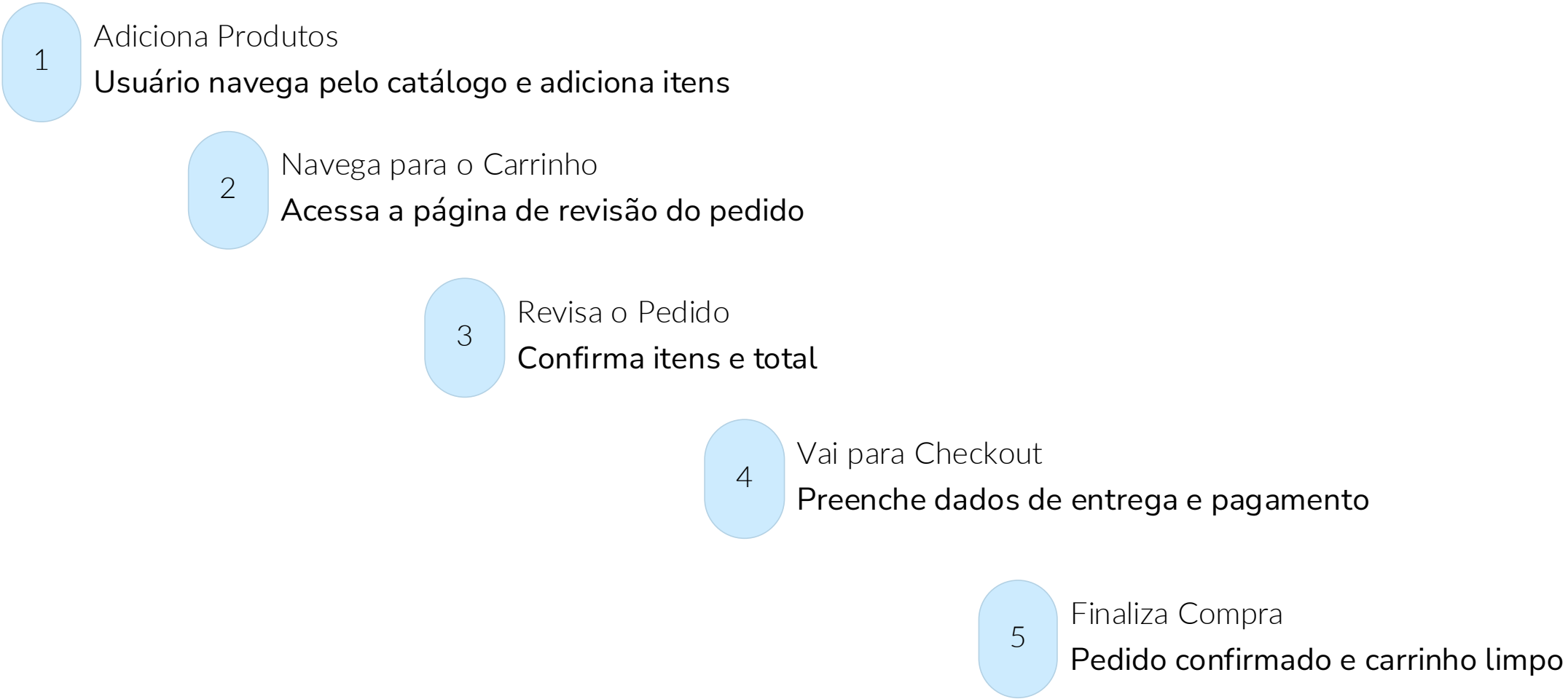
Sem padrão definido entre componentes

Objetivo da Refatoração

Após a refatoração, cada parte do sistema terá uma responsabilidade clara e bem definida, seguindo o padrão de e-commerce real.



Fluxo Correto do Usuário



Etapa 1 — Parar o Servidor

Antes de iniciar alterações estruturais em múltiplos arquivos, é essencial parar o servidor Angular para evitar erros de compilação durante as modificações.

Comando no Terminal

Pressione a combinação de teclas abaixo para encerrar o servidor:

Por que parar o servidor?

- Evitar erros durante alterações em múltiplos arquivos
- Prevenir recompilações parciais com código inconsistente
- Garantir que todas as mudanças sejam aplicadas de uma vez

CTRL + C



Certifique-se de que o servidor foi encerrado completamente antes de prosseguir com as próximas etapas.



Etapa 2 — Criar Modelo Compartilhado do Carrinho

O primeiro passo da refatoração é criar um **tipo global** para representar os itens do carrinho, evitando duplicação de estrutura de dados entre componentes.

Arquivo a Criar

Localização

`src/app/core/models/item-carrinho.ts`

Tipo de Arquivo

Tipo TypeScript compartilhado

Escopo

Módulo `core/models`. Acessível globalmente

Motivo

Evitar duplicação de estrutura de dados entre componentes. Com um modelo único e centralizado:

- Todos os componentes usam o mesmo tipo
- Mudanças no modelo refletem em todo o projeto
- TypeScript garante consistência em tempo de compilação

 Este arquivo será importado pelo `CarrinhoService`, `CarrinhoFacade` e pelo componente `Produto`.

Código — `item-carrinho.ts`

`src/app/core/models/item-carrinho.ts`

Tipo TypeScript que define a estrutura de um item no carrinho. Este modelo é compartilhado por todos os componentes do projeto.

```
// Tipo compartilhado para representar um item do carrinho.  
// Essa refatoração evita repetir o mesmo type em vários arquivos do projeto.  
  
export type ItemCarrinho = {  
  nome: string;  
  preco: number;  
};
```

- ✅ Com este tipo centralizado, qualquer componente que manipule itens do carrinho utilizará exatamente a mesma estrutura de dados, eliminando inconsistências.

Etapa 3 — Refatorar CarrinhoService

O `CarrinhoService` será transformado na **fonte única de verdade** do carrinho, centralizando todo o estado e lógica de manipulação.

Arquivo

`src/app/core/services/carrinho.service.ts`

Signals a Criar

- `itens` — lista de itens no carrinho
- `quantidade` — total de itens
- `total` — valor total da compra
- `carrinhoVazio` — estado vazio do carrinho

Métodos a Adicionar

`adicionar(item)`

Adiciona um produto ao carrinho global

`removerPorIndice(i)`

Remove um item específico pelo índice

`limpar()`

Esvazia completamente o carrinho

```
import { Injectable, computed, signal } from '@angular/core';

import { ItemCarrinho } from '../models/item-carrinho';

@Injectable({
  providedIn: 'root',
})
export class CarrinhoService {
  // O estado interno continua privado.
  // Apenas o service pode alterar diretamente a lista do carrinho.
  private carrinho = signal<ItemCarrinho[]>([]);

  // Selectors públicos derivados do estado interno.
  // Os componentes leem estes valores, mas não alteram diretamente o signal privado.
  itens = computed(() => this.carrinho());

  quantidade = computed(() => this.carrinho().length);

  total = computed(() =>
    this.carrinho().reduce((total, item) => total + item.preco, 0)
  );

  carrinhoVazio = computed(() => this.carrinho().length === 0);

  // Adiciona um produto ao carrinho global.
  adicionar(produto: ItemCarrinho) {
    this.carrinho.update((listaAtual) => [...listaAtual, produto]);
  }

  // Remove um item específico pelo índice.
  removerPorIndice(indice: number) {
    this.carrinho.update((listaAtual) =>
      listaAtual.filter((_, index) => index !== indice)
    );
  }

  // Limpa todos os itens do carrinho.
  limpar() {
    this.carrinho.set([]);
  }
}
```

Etapa 4 — Refatorar CarrinhoFacade

A CarrinhoFacade atua como camada intermediária entre os componentes e o serviço, **separando a lógica de negócio** do uso direto nos componentes.



Arquivo

`src/app/core/facades/carrinho.facade.ts`



Expor Dados

Disponibilizar os signals do serviço para os componentes de forma controlada



Métodos

`adicionarProduto`, `removerItem` e `limparCarrinho`

Padrão Facade: Os componentes nunca acessam o service diretamente. Sempre passam pela facade, que encapsula e coordena a lógica de negócio.

```
import { Injectable, inject } from '@angular/core';

import { CarrinhoService } from '../services/carrinho.service';
import { ItemCarrinho } from '../models/item-carrinho';

@Injectable({
  providedIn: 'root',
})
export class CarrinhoFacade {
  // A facade continua sendo a camada usada pelos componentes.
  // Os componentes não precisam conhecer os detalhes internos do CarrinhoService.
  private carrinhoService = inject(CarrinhoService);

  // Aula 32: sinais públicos de leitura usados pelo header, carrinho e checkout.
  itens = this.carrinhoService.itens;
  quantidade = this.carrinhoService.quantidade;
  total = this.carrinhoService.total;
  carrinhoVazio = this.carrinhoService.carrinhoVazio;

  // Ação de alto nível para adicionar produto.
  adicionarProduto(produto: ItemCarrinho) {
    this.carrinhoService.adicionar(produto);
  }

  // Ação de alto nível para remover item.
  removerItem(indice: number) {
    this.carrinhoService.removerPorIndice(indice);
  }

  // Ação de alto nível para limpar o carrinho.
  limparCarrinho() {
    this.carrinhoService.limpar();
  }
}
```

Etapa 5 — Atualizar Componente Produto

O componente `Produto` precisa ser atualizado para utilizar o tipo compartilhado `ItemCarrinho`, padronizando o envio de dados para o carrinho.

Arquivo

`src/app/features/produtos/produto/produto.ts`

Alterações Necessárias

- Substituir tipo local por `ItemCarrinho`
- Ajustar evento `produtoAdicionado`
- Importar o tipo `ItemCarrinho` do módulo `core/models`

Objetivo

Padronizar o envio de dados para o carrinho, garantindo que o evento emitido pelo componente de produto seja compatível com o tipo esperado pelo serviço e pela facade.

- ❗ Após esta alteração, o componente `Produto` emitirá um objeto do tipo `ItemCarrinho` ao invés de um tipo local.

Código — produto.ts

src/app/features/produtos/produto/produto.ts

```
import { CurrencyPipe } from '@angular/common';
import { Component, EventEmitter, Input, Output } from '@angular/core';

import { MatButtonModule } from '@angular/material/button';
import { MatCardModule } from '@angular/material/card';
import { ItemCarrinho } from '../../core/models/item-carrinho';

@Component({
  selector: 'app-produto',
  imports: [CurrencyPipe, MatButtonModule, MatCardModule],
  templateUrl: './produto.html',
  styleUrls: ['./produto.css'],
})
export class Produto {
  @Input() nome: string = '';
  @Input() preco: number = 0;

  @Output() produtoSelecionado = new EventEmitter<string>();

  // O evento agora usa o tipo compartilhado ItemCarrinho.
  @Output() produtoAdicionado = new EventEmitter<ItemCarrinho>();

  selecionarProduto() {
    this.produtoSelecionado.emit(this.nome);
  }

  adicionarAoCarrinho() {
    // O produto enviado ao carrinho segue o modelo compartilhado.
    this.produtoAdicionado.emit({
      nome: this.nome,
      preco: this.preco,
    });
  }
}
```

Etapa 6 — Limpar Lista de Produtos

A lista de produtos deve ser responsável **apenas pelo catálogo**. Toda lógica relacionada ao carrinho deve ser removida deste componente.

Remover do Componente

`quantidadeCarrinho`

Propriedade de contagem de itens

`totalCarrinho`

Propriedade de valor total

Lógica de Exibição

Qualquer exibição do estado do carrinho

Manter no Componente

- Lista de produtos do catálogo
- Envio de produto para o carrinho via facade
- Lógica de carregamento de produtos

Arquivo

`src/app/features/produtos/lista-produtos/lista-produtos.ts`

⚠ Objetivo: Separar completamente catálogo de carrinho. O componente de lista não deve exibir nem controlar o estado do carrinho; ele apenas envia produtos para a facade.

Código — lista-produtos.ts

```
import { Component, computed, effect, inject, signal } from '@angular/core';
import { MatButtonModule } from '@angular/material/button';
import { CarrinhoFacade } from '../../../core/facades/carrinho.facade';
import { ItemCarrinho } from '../../../core/models/item-carrinho';
import { ProdutosService } from '../../../core/services/produtos.service';
import { Produto } from '../../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto, MatButtonModule],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
export class ListaProdutos {
  produtosService = inject(ProdutosService);
  carrinhoFacade = inject(CarrinhoFacade);

  produtos = signal<{ nome: string; preco: number }[]>([]);
  produtoSelecionado = signal<string | null>(null);
  carregando = signal(true);
  erro = signal<string | null>(null);

  totalProdutos = computed(() => this.produtos().length);

  valorTotal = computed(() => {
    return this.produtos().reduce((total, item) => total + item.preco, 0);
  });

  constructor() {
    this.carregarProdutos();

    effect(() => {
      console.log('Lista de produtos alterada:', this.produtos());
    });
    effect(() => {
      console.log('Valor total atualizado:', this.valorTotal());
    });
    effect(() => {
      if (typeof document !== 'undefined') {
        document.title = `(${this.totalProdutos()}) Minha Loja`;
      }
    });
  }
}
```

src/app/features/produtos/lista-produtos/lista-produtos.ts

```
carregarProdutos() {
  this.erro.set(null);
  this.carregando.set(true);
  this.produtosService.buscarProdutos().subscribe({
    next: (dados) => {
      const produtos = this.produtosService.transformarProdutos(dados);
      this.produtos.set(produtos);
      this.carregando.set(false);
    },
    error: (erro) => {
      console.error('Erro ao carregar produtos:', erro);
      this.erro.set('Erro ao carregar produtos. Verifique sua conexão e tente novamente.');
```

```
      this.carregando.set(false);
    },
  });
}

exibirProduto(nome: string) {
  this.produtoSelecionado.set(nome);
}

adicionarProduto() {
  this.produtos.update((listaAtual) => [...listaAtual, { nome: 'Teclado', preco: 250 }]);
}

substituirProdutos() {
  this.produtos.set([{ nome: 'Produto novo', preco: 999 }]);
}

adicionarAoCarrinho(produto: ItemCarrinho) {
  this.carrinhoFacade.adicionarProduto(produto);
}
}
```

Etapa 7 — Remover Carrinho do HTML de Produtos

O template HTML da lista de produtos deve ser completamente limpo de qualquer referência ao carrinho, tornando-se uma página exclusiva de catálogo.

Remover Completamente

- Seção inteira do carrinho
- Listagem de itens do carrinho
- Exibição do total
- Botões de manipulação do carrinho
- Qualquer binding relacionado ao carrinho

Resultado Esperado

O arquivo `lista-produtos.html` deve conter apenas:

- Grid ou lista de produtos
- Componente `app-produto` para cada item
- Binding do evento `produtoAdicionado`

Arquivo:

```
src/app/features/produtos/lista-produtos/lista-produtos.html
```

Código — lista-produtos.html

```
<!-- Esta página volta a representar apenas o catálogo de produtos. -->
<section>
  <h2>Produtos:</h2>

  @if (carregando()) {
    <p>Carregando produtos...</p>
  }

  @if (erro()) {
    <p style="color: red">
      {{ erro() }}
    </p>

    <button (click)="carregarProdutos()">Tentar novamente</button>
  }

  @if (!carregando() && !erro()) {
    <p>Total de produtos: {{ totalProdutos() }}</p>

    <hr />

    <section class="grid-produtos">
      @for (item of produtos(); track $index) {
        <app-produto
          [nome]="item.nome"
          [preco]="item.preco"
          (produtoSelecioneado)="exibirProduto($event)"
          (produtoAdicionado)="adicionarAoCarrinho($event)"
        />
      }
    </section>
  }

  <hr />
</section>
```

src/app/features/produtos/lista-produtos/lista-produtos.html

```
<h2>
  Produto selecionado:
  @if (produtoSelecioneado()) {
    <strong style="color: green">{{ produtoSelecioneado() }}</strong>
  } @else {
    <span style="color: gray">Nenhum</span>
  }
</h2>

<button mat-raised-button color="primary" (click)="adicionarProduto()">
  Adicionar Produto
</button>

<button mat-stroked-button color="accent" (click)="substituirProdutos()">
  Substituir Lista
</button>

<h2>Total de produtos: {{ totalProdutos() }}</h2>
<h3>Valor total: R$ {{ valorTotal().toFixed(2) }}</h3>
</section>
```

Etapa 8 — Implementar Página Carrinho

A página de carrinho será transformada na **tela oficial de revisão do pedido**, utilizando a `CarrinhoFacade` para acessar e manipular os dados.

Arquivo

`src/app/features/carrinho/carrinho/carrinho.ts`

Adicionar ao Componente

- Injeção da `CarrinhoFacade`
- Método `removeItem(indice)`
- Método `limparCarrinho()`
- Acesso aos signals do carrinho

Objetivo

Transformar a página carrinho na tela oficial do pedido, onde o usuário pode:

- Revisar itens
 - Ver todos os produtos adicionados
- Remover produtos
 - Excluir itens indesejados
- Prosseguir
 - Navegar para o checkout

Código — carrinho.ts

src/app/features/carrinho/carrinho/carrinho.ts

```
import { Component, inject } from '@angular/core';
import { RouterLink } from '@angular/router';

import { MatButtonModule } from '@angular/material/button';

import { CarrinhoFacade } from '../../../core/facades/carrinho.facade';

@Component({
  selector: 'app-carrinho',
  imports: [RouterLink, MatButtonModule],
  templateUrl: './carrinho.html',
  styleUrls: ['./carrinho.css'],
})
export class Carrinho {
  // A página do carrinho passa a consumir o estado global do carrinho.
  carrinhoFacade = inject(CarrinhoFacade);

  removerItem(indice: number) {
    // Remove um item específico da lista.
    this.carrinhoFacade.removerItem(indice);
  }

  limparCarrinho() {
    // Limpa todos os itens do carrinho.
    this.carrinhoFacade.limparCarrinho();
  }
}
```

Etapa 9 – Implementar Template do Carrinho

O template HTML do carrinho deve ser uma tela completa e funcional de revisão de pedido, com todas as interações necessárias para o usuário.



Listagem de Itens

Exibir todos os produtos com nome e preço



Botão Remover

Permitir remoção individual de cada item



Total do Carrinho

Exibir valor total calculado automaticamente



Limpar Carrinho

Botão para esvaziar todos os itens de uma vez



Navegação

Continuar comprando ou ir para o checkout



Arquivo: `src/app/features/carrinho/carrinho/carrinho.html`

Código — carrinho.html

src/app/features/carrinho/carrinho/carrinho.html

```
<!-- A página /carrinho passa a ser a tela oficial de revisão do pedido. -->
<section class="pagina-carrinho">
  <h1>Carrinho</h1>

  @if (carrinhoFacade.carrinhoVazio()) {
    <p>Seu carrinho está vazio.</p>

    <button mat-raised-button color="primary" routerLink="/produtos">
      Ver produtos
    </button>
  } @else {
    <section class="lista-carrinho">
      @for (item of carrinhoFacade.itens(); track $index) {
        <article class="item-carrinho">
          <div>
            <h3>{{ item.nome }}</h3>
            <p>R$ {{ item.preco.toFixed(2) }}</p>
          </div>

          <button
            mat-stroked-button
            color="warn"
            type="button"
            (click)="removeItem($index)"
          >
            Remover
          </button>
        </article>
      }
    </section>

    <hr />

    <section class="resumo-carrinho">
      <h3>Itens no carrinho: {{ carrinhoFacade.quantidade() }}</h3>
      <h2>Total: R$ {{ carrinhoFacade.total().toFixed(2) }}</h2>
    </section>
  }
</section>
```

```
<section class="acoes-carrinho">
  <button mat-button type="button" (click)="limparCarrinho()">
    Limpar carrinho
  </button>

  <button mat-stroked-button color="primary" routerLink="/produtos">
    Continuar comprando
  </button>

  <button mat-raised-button color="primary" routerLink="/checkout">
    Ir para checkout
  </button>
</section>
}
```

Etapa 10 – Ajustar Checkout

O checkout deve ser responsável **apenas pela finalização da compra**. Toda duplicação do carrinho deve ser removida, mantendo apenas um resumo do pedido.

Remover do Checkout

- Duplicação completa do carrinho
- Botões de manipulação de itens
- Lógica de remoção de produtos
- Qualquer funcionalidade já presente na página de carrinho

Adicionar ao Checkout

Resumo do Pedido

Exibição somente leitura do resumo do pedido: quantidade e total

Botão "Revisar Carrinho"

Navegar de volta para a página de carrinho

Formulário de Finalização

Dados de entrega e pagamento

Código — checkout.html

src/app/features/checkout/checkout/checkout.html

```
<h2>Checkout</h2>
@if (compraFinalizada()) {
  <p style="color: green">
    Compra finalizada com sucesso!
  </p>
  <button type="button" routerLink="/produtos">
    Voltar aos produtos
  </button>
}
@if (carrinhoFacade.carrinhoVazio() && !compraFinalizada()) {
  <p style="color: red">
    Seu carrinho está vazio. Adicione produtos antes de finalizar a compra.
  </p>
  <button type="button" routerLink="/produtos">
    Ver produtos
  </button>
} @else if (!compraFinalizada()) {
  <section>
    <h3>Resumo do pedido</h3>
    <p>
      Itens no carrinho: {{ carrinhoFacade.quantidade() }}
    </p>
    <h3>Total: R$ {{ carrinhoFacade.total().toFixed(2) }}</h3>
    <button type="button" routerLink="/carrinho">
      Revisar carrinho
    </button>
  </section>
  <hr />
  <form [formGroup]="formulario">
    <p>
      <input type="text" formControlName="nome" placeholder="Nome" />
    </p>
```

```
@if (formulario.get('nome')?.touched && formulario.get('nome')?.invalid) {
  @if (formulario.get('nome')?.errors?.['required']) {
    <small>Nome é obrigatório</small>
  }
  @if (formulario.get('nome')?.errors?.['minlength']) {
    <small>Nome deve ter no mínimo 3 caracteres</small>
  }
  @if (formulario.get('nome')?.errors?.['numeroInvalido']) {
    <small>Nome não pode conter números</small>
  }
}
<p><input type="email" formControlName="email" placeholder="E-mail" /></p>
@if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
  @if (formulario.get('email')?.errors?.['required']) {
    <small>Email é obrigatório</small>
  }
  @if (formulario.get('email')?.errors?.['email']) {
    <small>Email inválido</small>
  }
}
<p><input type="text" formControlName="endereco" placeholder="Endereço" /></p>
@if (formulario.get('endereco')?.touched && formulario.get('endereco')?.invalid) {
  @if (formulario.get('endereco')?.errors?.['required']) {
    <small>Endereço é obrigatório</small>
  }
  @if (formulario.get('endereco')?.errors?.['minlength']) {
    <small>Endereço deve ter no mínimo 5 caracteres</small>
  }
}
<p>
  <button type="button" (click)="finalizar()" [disabled]="formulario.invalid">Finalizar Compra</button>
</p>
</form>
}
```

Etapa 11 — Ajustar Lógica do Checkout

O componente TypeScript do checkout precisa garantir o **fluxo correto de finalização**, validando o carrinho e limpando-o após a conclusão da compra.

Arquivo

`src/app/features/checkout/checkout/checkout.ts`

Alterações Necessárias

- Validar se o carrinho está vazio antes de finalizar a compra
- Exibir mensagem orientando o usuário a voltar para produtos quando o carrinho estiver vazio
- Limpar carrinho após finalizar compra
- Manter formulário reativo com validações

Fluxo de Finalização

01

Verificar Carrinho

Se vazio, impedir finalização e orientar o usuário

02

Validar Formulário

Garantir dados obrigatórios preenchidos

03

Processar Pedido

Enviar dados e confirmar compra

04

Limpar Carrinho

Esvaziar após confirmação

Código — checkout.ts

```
import { Component, inject, signal } from '@angular/core';
import {
  AbstractControl,
  FormControl,
  FormGroup,
  ReactiveFormsModule,
  ValidationErrors,
  Validators,
} from '@angular/forms';
import { RouterLink } from '@angular/router';

import { CarrinhoFacade } from '../../core/facades/carrinho.facade';

function nomeSemNumeros(control: AbstractControl): ValidationErrors | null {
  const valor = control.value;

  if (!valor) return null;

  if (/\\d/.test(valor)) {
    return { numeroInvalido: true };
  }

  return null;
}

@Component({
  selector: 'app-checkout',
  imports: [ReactiveFormsModule, RouterLink],
  templateUrl: './checkout.html',
  styleUrls: ['./checkout.css'],
})
export class Checkout {
  // Checkout usa a facade apenas para ler resumo, validar carrinho vazio e limpar após compra.
  carrinhoFacade = inject(CarrinhoFacade);

  compraFinalizada = signal(false);

  formulario = new FormGroup({
    nome: new FormControl('', [Validators.required, Validators.minLength(3), nomeSemNumeros]),
    email: new FormControl('', [Validators.required, Validators.email]),
    endereco: new FormControl('', [Validators.required, Validators.minLength(5)]),
  });
```

src/app/features/checkout/checkout/checkout.ts

```
finalizar() {
  this.compraFinalizada.set(false);

  if (this.carrinhoFacade.carrinhoVazio()) {
    console.log('Não é possível finalizar uma compra com o carrinho vazio.');
    return;
  }

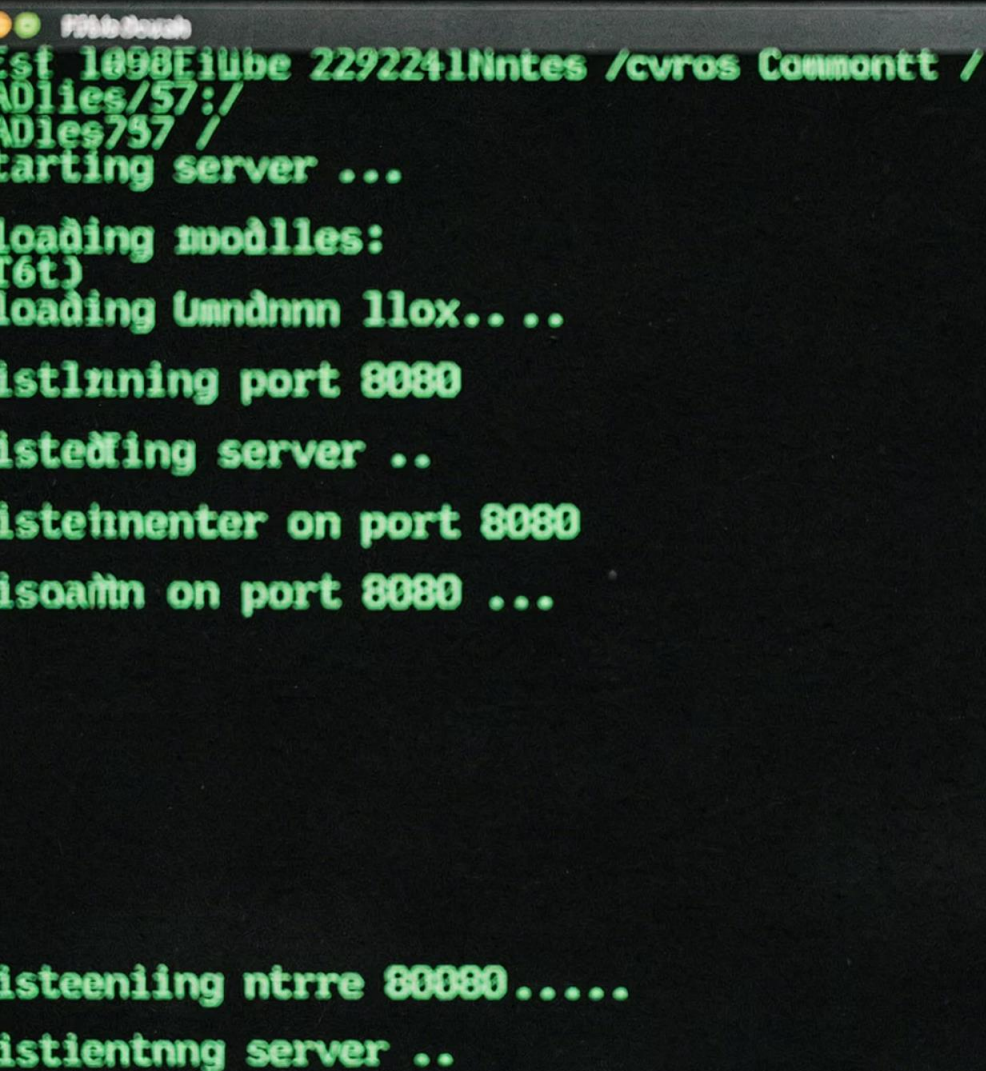
  if (this.formulario.invalid) {
    console.log('Formulário inválido');
    this.formulario.markAllAsTouched();
    return;
  }

  const dados = this.formulario.value;
  const itens = this.carrinhoFacade.itens();
  const total = this.carrinhoFacade.total();

  console.log('Compra finalizada com sucesso!');
  console.log('Dados do formulário:', dados);
  console.log('Itens do carrinho:', itens);
  console.log('Total da compra:', total);

  // Após finalizar, o carrinho global é limpo.
  this.carrinhoFacade.limparCarrinho();

  this.formulario.reset();
  this.compraFinalizada.set(true);
}
```



```
st 1098EiUbe 2292241Nntes /cvros Commontt /  
ADlies/57:/  
ADlies757 /  
starting server ...  
loading modules:  
(6t)  
loading Umndnnn llox... ..  
listening port 8080  
starting server ..  
listening on port 8080  
listening on port 8080 ...  
listening ntrre 80080.....  
starting server ..
```

Etapa 12 - Reiniciar a Aplicação

Com todas as alterações realizadas, é hora de reiniciar o servidor Angular e verificar se a aplicação está funcionando corretamente.

Comando no Terminal

```
ng serve
```

Acessar no Navegador

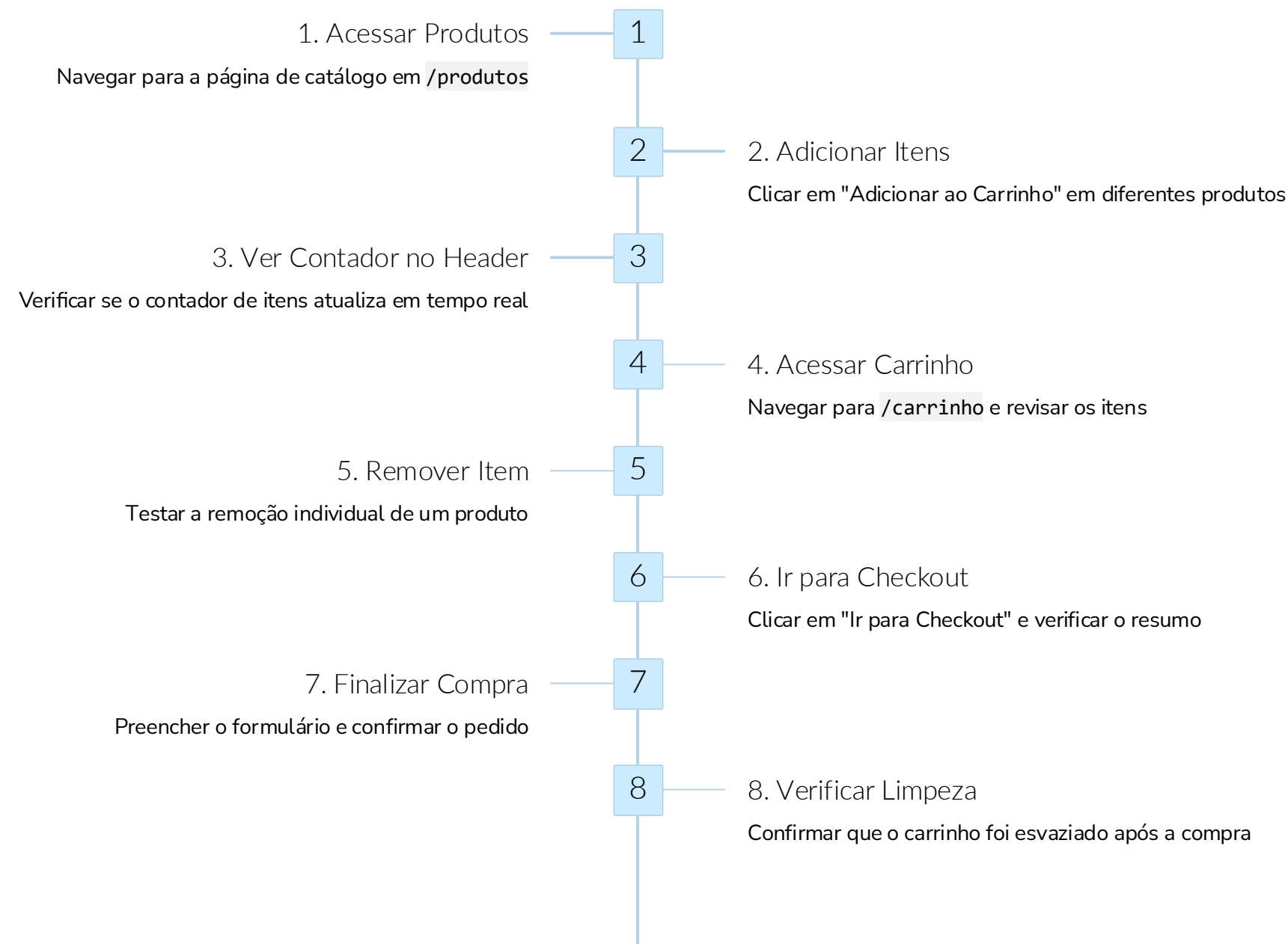
```
http://localhost:4200
```

Aguarde a compilação completa antes de testar o fluxo.

-  Se houver erros de compilação, verifique os imports de `ItemCarrinho` em todos os componentes alterados e confirme que os caminhos estão corretos.

Teste Completo do Fluxo

Após reiniciar a aplicação, realize o teste completo do fluxo de e-commerce para validar que a refatoração foi bem-sucedida.



Resultado da Refatoração

Após a conclusão de todas as etapas, o sistema agora possui uma arquitetura sólida e profissional.



Estado Centralizado

Todo o estado do carrinho gerenciado pelo `CarrinhoService` com signals reativos



Responsabilidades Bem Definidas

Cada componente com papel único: catálogo, revisão e finalização



Fluxo Real de E-commerce

Catálogo → Carrinho → Checkout funcionando como aplicação de mercado



Código Limpo e Escalável

Arquitetura preparada para crescimento e novas funcionalidades



Conclusão da Aula

Nesta aula você aplicou conceitos fundamentais de arquitetura de software que são utilizados em **aplicações reais de mercado**.

- 1 Separação de Responsabilidades
Cada componente, service e facade com papel bem definido no sistema
- 2 Centralização de Estado
Uso de signals para gerenciar o estado do carrinho de forma reativa e consistente
- 3 Uso de Services e Facades
Padrão de arquitetura que separa lógica de negócio da camada de apresentação
- 4 Organização Profissional
Estrutura de pastas e módulos seguindo as melhores práticas do ecossistema Angular

✓ **Este é o padrão utilizado em aplicações reais de mercado.** Você agora possui uma base sólida para construir e escalar projetos Angular com qualidade profissional.



Dúvidas

?

